SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

CO2 - ASSESSMENT TOOL - 3

Error Analysis Task

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO2**	**Assessment**	**Assessment Tool 3: Error Analysis Task**
Weightage	**50% of CIA**	**Total Marks**	**100**
CO2 Statement	**Design effective test cases using black-box, white-box testing techniques and to control methods flow.**		
Student Name	**B.Thejaswini**	**Reg. No.**	**192412479**
Section/Batch	**Slot A /2024**	**Date**	**04-11-2026**

Code of Conduct:

I,B.Thejaswini, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

B.Thejaswini

1. INTRODUCTION AND SYSTEM OVERVIEW

1.1 Introduction

Software testing is a systematic activity used to evaluate whether a software application satisfies specified requirements and to identify defects before the application is released. Testing improves product quality, reliability, security and user satisfaction. It involves preparing test conditions, selecting test data, executing test cases, comparing actual and expected results, and reporting defects.

An online shopping system enables customers to create accounts, log in, search for products, view product details, add products to a cart, select quantities, enter delivery information, make payments and receive order confirmation. Each function must be tested with normal, invalid and exceptional inputs.

1.2 Problem statement

A tester is designing test cases for the quantity field in an online shopping system. The tester uses Equivalence Partitioning but incorrectly places valid and invalid values in one class. The tester also ignores values at and around the minimum and maximum limits. The task is to identify these errors, explain their root causes, create corrected test cases using Equivalence Partitioning and Boundary Value Analysis, and show how black-box, white-box and control-flow methods improve defect detection.

1.3 Assumptions for the example

To make the test design measurable, the following requirement is assumed for one product: a customer may order a whole-number quantity from 1 to 10, and the available stock is 10 units. The quantity must be a positive integer and must not exceed available stock. The same principle can be adapted when the actual application has a different stock value or business rule.

Requirement ID	Requirement / rule	Expected behaviour
QTY-01	Quantity must be an integer.	Reject letters, decimals and non-numeric values.
QTY-02	Minimum quantity is 1.	Accept 1; reject values below 1.
QTY-03	Maximum quantity is 10 for this test scenario.	Accept 10; reject values above 10.

Requirement ID	Requirement / rule	Expected behaviour
QTY-04	Quantity field is mandatory.	Reject blank or missing input.
QTY-05	Accepted quantity must be available in stock.	Do not add more units than available stock.

1.4 Testing objectives

- Identify valid and invalid input classes without mixing their expected behaviours.
- Test values at the minimum and maximum boundaries and immediately outside them.
- Verify the observable behaviour of the shopping application using black-box techniques.
- Verify important program statements, decisions and execution paths using white-box techniques.
- Improve defect detection while avoiding unnecessary repetition of equivalent test data.

2. ANALYSIS OF THE GIVEN TEST-DESIGN PROBLEM

2.1 Main errors identified

No.	Error in the original design	Why it is a problem
1	Valid and invalid values are grouped in one equivalence class.	The class does not represent one expected behaviour, so a selected test value may hide failures affecting other values.
2	Minimum and maximum limits are not identified.	The tester cannot determine which values must be accepted or rejected.
3	Values just below and above the limits are omitted.	Off-by-one defects and incorrect comparison operators may remain undetected.
4	Only typical values may be selected.	Testing a normal value such as 5 does not prove that 1 and 10 are handled correctly.

No.	Error in the original design	Why it is a problem
5	Input formats are not considered.	Blank, alphabetic and decimal inputs may cause validation or conversion errors.
6	Expected results are not clearly specified.	The tester cannot reliably decide whether the observed behaviour is correct.

2.2 Root-cause analysis

Root cause	Reasoning	Corrective action
Incomplete requirement analysis	The tester has not translated the quantity rule into explicit minimum, maximum and data-type constraints.	Review the requirement and record the permitted range and input format before designing tests.
Misunderstanding of equivalence classes	Equivalence Partitioning requires each class to contain inputs expected to be handled in the same way.	Separate accepted quantities from each category of rejected quantities.
Lack of boundary-focused thinking	The tester has selected ordinary values but ignored the edges of the valid range.	Identify min, min−1, min+1, max−1, max and max+1.
Insufficient test review	No checklist or peer review has detected the missing classes and boundaries.	Review test conditions against requirements and use a coverage checklist.
Unclear expected results	Without an expected result, pass/fail decisions may be inconsistent.	Write an observable expected result for every test case.

Reasoning: A test class is useful only when its members are expected to produce the same category of result. If valid and invalid quantities are mixed, one representative value cannot reliably represent the entire class. Similarly, testing only values well inside the valid range cannot expose errors at the edges. The original design therefore has weak input-domain coverage.

3. EQUIVALENCE PARTITIONING: ERRORS AND CORRECTED CLASSES

3.1 Definition

Equivalence Partitioning (EP) is a black-box test-design technique in which the input domain is divided into groups called equivalence partitions or equivalence classes. Values within one class are expected to be processed in a similar way. A representative value is selected from each class instead of testing every possible value.

3.2 Correct classification for the quantity field

Class ID	Input class	Example representative	Classification	Expected result
EC-01	Whole-number quantity less than 1 (within numeric input domain)	0	Invalid	Reject and display quantity validation message.
EC-02	Whole-number quantity from 1 through 10	5	Valid	Accept quantity and allow it to be added to the cart, subject to stock.
EC-03	Whole-number quantity greater than 10	11	Invalid	Reject and display maximum/stock message.
EC-04	Blank or missing quantity	Blank	Invalid	Display a required-field message.
EC-05	Alphabetic or non-numeric text	ABC	Invalid	Display an invalid quantity message; application must not crash.
EC-06	Decimal quantity	2.5	Invalid	Reject because quantity must be a whole number.

Note: EC-01 to EC-03 describe numeric whole-number ranges. EC-04 to EC-06 describe different input-format classes. In an implementation, the tester should also consider signs, whitespace and very large values if these are permitted by the interface or specified in the requirements.

3.3 Corrected Equivalence Partitioning test cases

Test case ID	Test data	Class	Test action	Expected result
EP-01	0	EC-01	Enter 0 and select Add to Cart.	Input is rejected; quantity must be at least 1.

Test case ID	Test data	Class	Test action	Expected result
EP-02	5	EC-02	Enter 5 and select Add to Cart.	Quantity is accepted; cart shows 5 units.
EP-03	11	EC-03	Enter 11 and select Add to Cart.	Input is rejected; quantity exceeds available stock/limit.
EP-04	Blank	EC-04	Leave the field empty and submit.	Required-field validation is displayed.
EP-05	ABC	EC-05	Enter alphabetic characters and submit.	Invalid-input message is displayed; no crash occurs.
EP-06	2.5	EC-06	Enter a decimal quantity and submit.	Decimal quantity is rejected.

3.4 Why the corrected classification is effective

- Every class has a clear expected behaviour: accepted or rejected.
- One representative value can provide useful coverage of each class.
- Different invalid formats are tested separately because they may fail for different reasons.
- The approach reduces redundant tests while retaining coverage of distinct input behaviours.
- Explicit class identifiers make it easier to trace requirements to test cases and defects.

4. BOUNDARY VALUE ANALYSIS AND CORRECTED TEST CASES

4.1 Definition

Boundary Value Analysis (BVA) is a black-box technique that focuses on values at the edges of an input range. Defects often occur where a program changes from accepting to rejecting data. For an inclusive range from 1 to 10, the key values are 0, 1, 2, 9, 10 and 11.

4.2 Boundary identification

Boundary position	Formula	Value	Expected result
Immediately below minimum	Minimum − 1	0	Reject
Minimum boundary	Minimum	1	Accept
Immediately above minimum	Minimum + 1	2	Accept

Boundary position	Formula	Value	Expected result
Immediately below maximum	Maximum − 1	9	Accept
Maximum boundary	Maximum	10	Accept
Immediately above maximum	Maximum + 1	11	Reject

4.3 Boundary Value Analysis test cases

Test case ID	Input quantity	Boundary checked	Expected result	Purpose
BVA-01	0	Minimum − 1	Reject	Checks lower out-of-range value.
BVA-02	1	Minimum	Accept	Checks inclusive lower limit.
BVA-03	2	Minimum + 1	Accept	Checks value just inside lower limit.
BVA-04	9	Maximum − 1	Accept	Checks value just inside upper limit.
BVA-05	10	Maximum	Accept	Checks inclusive upper limit.
BVA-06	11	Maximum + 1	Reject	Checks upper out-of-range value.

4.4 Defects detected by boundary testing

Possible defect	Example faulty logic	Test that can reveal it
Lower limit is too strict	quantity > 1 instead of quantity >= 1	BVA-02: quantity 1 is incorrectly rejected.
Upper limit is too strict	quantity < 10 instead of quantity <= 10	BVA-05: quantity 10 is incorrectly rejected.

Possible defect	Example faulty logic	Test that can reveal it
Lower invalid value is accepted	Validation does not reject 0	BVA-01: quantity 0 is incorrectly accepted.
Upper invalid value is accepted	Validation does not reject 11	BVA-06: quantity 11 is incorrectly accepted.

The six-value set is a common two-value boundary test set for one numeric range. If the application has multiple fields, each field needs its own boundary analysis. If the stock changes dynamically, the maximum boundary must be based on the current available stock rather than a fixed value of 10.

4.5 Equivalence Partitioning and BVA: difference

Aspect	Equivalence Partitioning	Boundary Value Analysis
Main focus	Groups of inputs expected to behave similarly.	Edges of valid and invalid ranges.
Test selection	Representative value from each class.	Values at and near each boundary.
Example	5 represents the valid class 1–10.	0, 1, 2, 9, 10 and 11.
Best at detecting	Errors affecting a category of inputs or formats.	Off-by-one and range-comparison errors.

5. BLACK-BOX TESTING FOR THE ONLINE SHOPPING SYSTEM

5.1 Definition

Black-Box Testing evaluates the externally visible behaviour of a system without requiring the tester to inspect its source code. The tester supplies inputs, performs user actions and compares the actual output with the expected output. EP and BVA are commonly used black-box test-design techniques.

5.2 Main functional areas

Module	What should be tested
Registration	Required fields, valid email format, password rules and duplicate accounts.

Module	What should be tested
Login	Valid credentials, invalid credentials, blank fields and account access.
Product search	Existing product, unavailable product, empty search and partial product name.
Product details	Correct name, price, description and stock information.
Shopping cart	Add item, remove item, update quantity and calculate subtotal.
Delivery details	Required address fields, valid postal code and missing information.
Payment	Valid payment, declined payment, cancelled payment and duplicate submission.
Order confirmation	Order is confirmed only after successful payment and required checks.

5.3 Black-box test cases

ID	Functionality	Input / action	Expected output
BB-01	Registration	Enter valid name, email and password.	Account is created or registration success is shown.
BB-02	Registration	Enter malformed email address.	Validation message is displayed; account is not created.
BB-03	Login	Enter correct username and password.	User is logged in and taken to the appropriate page.
BB-04	Login	Enter an incorrect password.	Login is denied and an appropriate message is shown.
BB-05	Product search	Search for an existing product.	Matching product is displayed.
BB-06	Product search	Search for a product that does not exist.	No-results message is displayed.
BB-07	Add to cart	Select an available product and add it.	Product appears in the cart with correct details.
BB-08	Quantity validation	Enter quantity 5.	Quantity is accepted if stock is at least 5.

ID	Functionality	Input / action	Expected output
BB-09	Quantity validation	Enter quantity 0.	Quantity is rejected.
BB-10	Quantity validation	Enter quantity 11 when stock is 10.	Quantity is rejected.
BB-11	Delivery	Enter all required valid delivery details.	Delivery details are accepted.
BB-12	Delivery	Leave a required address field blank.	Required-field message is displayed.
BB-13	Payment	Complete payment successfully.	Payment status is successful and order confirmation is generated.
BB-14	Payment	Use a simulated declined/cancelled payment.	Order is not marked as paid; a suitable message is displayed.

5.4 Advantages of black-box testing

- It validates the application from the customer's point of view.
- It does not require knowledge of the internal implementation.
- It checks whether the system satisfies functional requirements.
- It is useful for detecting missing functions, incorrect outputs and input-validation problems.
- It supports systematic test design through EP, BVA, decision tables and state-transition testing.

6. WHITE-BOX AND CONTROL-FLOW TESTING

6.1 Definition of White-Box Testing

White-Box Testing examines the internal structure of the software, including statements, branches, conditions, loops and execution paths. The tester needs access to the code or a sufficiently detailed design. It complements black-box testing: black-box tests check what the application does, while white-box tests help verify how the implementation reaches its result.

6.2 Example quantity-validation pseudocode

```
quantity_input = read_input()
stock = 10

if input_is_blank(quantity_input):
    display("Quantity is required")
elif not is_integer(quantity_input):
    display("Enter a whole-number quantity")
else:
    quantity = convert_to_integer(quantity_input)
```

```
if quantity < 1:
    display("Quantity must be at least 1")
elif quantity > stock:
    display("Quantity exceeds available stock")
else:
    total = quantity * 100
    add_to_cart(quantity, total)
    display("Product added to cart")
```

6.3 Explanation of program logic

1. The application first reads the quantity entered by the customer.
2. It checks whether the field is blank. If it is blank, the application displays a required-field message.
3. It checks whether the input is a whole number. Letters and decimal values are rejected before conversion.
4. If the input is an integer, the application checks whether it is below 1.
5. If it is not below 1, the application checks whether it exceeds available stock.
6. If both range checks pass, the application calculates the total and adds the item to the cart.

6.4 White-box test cases

ID	Input	Internal statement / decision exercised	Expected result
WB-01	Blank	Blank-input decision: true branch.	Required-field message.
WB-02	ABC	Integer-format decision: false branch.	Whole-number validation message.
WB-03	0	Lower-bound decision: quantity < 1 is true.	Minimum quantity message.
WB-04	11	Upper-bound decision: quantity > stock is true.	Stock-limit message.
WB-05	5	Both range decisions are false; valid processing executes.	Item is added; total is 500.
WB-06	1	Valid path at the lower inclusive boundary.	Item is added; total is 100.
WB-07	10	Valid path at the upper inclusive boundary.	Item is added; total is 1000.

6.5 Control-Flow Testing

Control-Flow Testing is a white-box technique that designs tests to exercise the possible flow of control through a program. A Control Flow Graph (CFG) represents program execution. Nodes represent

statements or blocks of statements, and directed edges represent the possible transfer of control. Decision nodes have alternative outgoing edges, such as True and False.

6.6 Text representation of the control-flow graph

Node	Program action / decision	Possible next node
N1	Start and read quantity input	N2
N2	Is input blank?	Yes → N3; No → N4
N3	Display required-field message	N9
N4	Is input a whole number?	No → N5; Yes → N6
N5	Display whole-number error	N9
N6	Is quantity less than 1?	Yes → N7; No → N8
N7	Display minimum quantity error	N9
N8	Is quantity greater than stock?	Yes → N10; No → N11
N10	Display stock-limit error	N9
N11	Calculate total and add item to cart	N12
N12	Display success message	N9
N9	End	—

The graph can be drawn from the node table: place N1 at the top, connect the decision nodes in sequence, draw separate error branches to the corresponding message nodes, and connect all outcomes to the End node. This gives a clear visual representation of the program's control flow.

6.7 Independent execution paths

Path ID	Path through the logic	Representative input	Expected result
P1	N1 → N2 (Yes) → N3 → N9	Blank	Required-field message.
P2	N1 → N2 (No) → N4 (No) → N5 → N9	ABC	Whole-number error.
P3	N1 → N2 (No) → N4 (Yes) → N6 (Yes) → N7 → N9	0	Minimum quantity error.
P4	N1 → N2 (No) → N4 (Yes) → N6 (No) → N8 (Yes) → N10 → N9	11	Stock-limit error.
P5	N1 → N2 (No) → N4 (Yes) → N6 (No) → N8 (No) → N11 → N12 → N9	5	Item added; total calculated.

6.8 Coverage considerations

Statement coverage checks whether each executable statement has been run at least once. Branch coverage checks whether each decision outcome (True and False) has been exercised. Path coverage checks execution paths. Complete path coverage may be impractical in programs with many branches or loops, so the test objective should be selected according to risk and project requirements.

Coverage type	Meaning	Example in this scenario
Statement coverage	Execute every relevant statement at least once.	Run tests that reach each validation message and the successful cart operation.
Branch coverage	Exercise each decision outcome.	Test blank/non-blank, integer/non-integer, below-minimum/not-below, over-stock/within-stock.
Path coverage	Exercise selected independent paths through the logic.	Execute P1 to P5 shown above.

6.9 Advantages of white-box and control-flow testing

- Helps reveal errors in conditions, calculations and internal processing.
- Shows whether both outcomes of important decisions have been tested.
- Can reveal unreachable statements or missing branches.
- Supports measurable statement and branch coverage.
- Complements black-box testing by examining internal logic that may not be obvious from user-interface testing.

7. DEFECT DETECTION EFFECTIVENESS AND RECOMMENDATIONS

7.1 How the corrected design improves defect detection

Correction applied	Defect type more likely to be detected	Benefit
Separate valid and invalid equivalence classes	Incorrect acceptance or rejection of a category of inputs.	Improves input-domain coverage and makes results easier to interpret.
Test blank, text and decimal inputs separately	Missing required-field checks, unsafe conversion and incorrect data-type validation.	Prevents invalid data from causing errors or application crashes.
Test min, min−1 and min+1	Lower-bound comparison errors and off-by-one defects.	Confirms the lower limit is inclusive and values below it are rejected.

Correction applied	Defect type more likely to be detected	Benefit
Test max, max−1 and max+1	Upper-bound comparison errors and stock-limit defects.	Confirms the upper limit is inclusive and over-limit values are rejected.
Apply black-box tests to shopping functions	Missing or incorrect user-visible functionality.	Checks the complete customer workflow.
Apply white-box and control-flow tests	Untested branches, incorrect conditions and faulty calculations.	Provides visibility into internal logic and decision outcomes.

7.2 Example of a boundary defect

Suppose the developer accidentally writes the condition quantity > 1 and quantity < 10. This implementation rejects 1 and 10 even though the requirement says both limits are allowed. A normal test using quantity 5 would pass and would not reveal the defect. BVA tests using 1 and 10 expose the problem immediately.

Correct logic for the stated range is:

1 <= quantity <= 10

7.3 Recommended test execution sequence

7. Review the requirements and confirm the quantity range, data type and stock rules.
8. Prepare equivalence classes for valid numeric values and each relevant invalid input category.
9. Select representative values and record expected results.
10. Identify all numeric boundaries and prepare the BVA test set.
11. Execute black-box tests through the shopping user interface or API.
12. Review the implementation and prepare white-box tests for statements and decision branches.
13. Execute control-flow paths and record coverage achieved.
14. Log each failure with test data, steps, expected result, actual result, severity and supporting evidence.
15. Retest corrected defects and perform regression testing on related cart and checkout functions.

7.4 Sample defect report

Field	Sample content
Defect ID	BUG-QTY-001
Title	Maximum allowed quantity is rejected
Module	Shopping Cart – Quantity Validation
Precondition	Product is available with stock of 10 units.

Field	Sample content
Steps to reproduce	Open product page; enter quantity 10; select Add to Cart.
Expected result	Quantity 10 is accepted and the cart displays 10 units.
Actual result	Application displays an invalid quantity message.
Severity / priority	Medium / High (subject to project impact and team policy).
Related test case	BVA-05
Status	Open – pending developer investigation.

8. CONCLUSION

The original test design is incomplete because it combines valid and invalid values in an unsuitable equivalence class and does not test the minimum and maximum boundaries. These mistakes can allow input-validation defects, incorrect comparison operators and stock-limit errors to remain undetected.

The corrected approach separates the input domain into clear equivalence classes and tests representative values from each class. Boundary Value Analysis checks values immediately below, at and immediately above the valid limits. Black-box testing verifies the observable shopping functions, while white-box and control-flow testing examine internal statements, decisions and execution paths.

Combining these techniques provides more systematic test coverage, improves the chance of finding defects and gives the testing team clear evidence that the quantity-validation logic behaves according to the requirements. The same method can be extended to prices, discounts, delivery fields, payment limits and other input fields in an online shopping application.

9. REFERENCES

1. International Software Testing Qualifications Board (ISTQB). (2024). Certified Tester Foundation Level (CTFL) Syllabus, Version 4.0.1. https://www.istqb.org/

2. Myers, G. J., Sandler, C., & Badgett, T. (2011). The Art of Software Testing (3rd ed.). John Wiley & Sons.

3. Jorgensen, P. C. (2013). Software Testing: A Craftsman's Approach (4th ed.). CRC Press.

4. Ammann, P., & Offutt, J. (2016). Introduction to Software Testing (2nd ed.). Cambridge University Press.

5. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.

6. Kaner, C., Falk, J., & Nguyen, H. Q. (1999). Testing Computer Software (2nd ed.). John Wiley & Sons.

7. Beizer, B. (1990). Software Testing Techniques (2nd ed.). Van Nostrand Reinhold.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Error Identification	20%	Accurately identifies all errors with clear understanding of issues.	Identifies most errors with minor omissions.	Identifies limited errors; some are missed.	Fails to identify key errors.	_____
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning.	Explains causes with moderate clarity.	Partial explanation; weak reasoning.	Unable to identify causes of errors.	_____
Correction & Justification	25%	Provides correct solutions with strong justification and logic.	Provides correct corrections with some justification.	Corrections are partially correct or weakly justified.	Incorrect or no corrections provided.	_____
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors.	Applies concepts with minor errors.	Limited application; requires guidance.	Unable to apply concepts.	_____
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically.	Generally clear with minor issues.	Lacks clarity or proper structure.	Poor presentation and difficult to understand.	_____